UNIVERSIDADE CATÓLICA DE BRASÍLIA

CURSO: ENGENHARIA DE SOFTWARE

DISCIPLINA: REQUISITOS DE SOFTWARE / ARQUITETURA DE SOFTWARE

ARTHUR DUARTE LEMES

CAUÃ FREITAS VIEIRA

DAVI CAMPANARO SILVA

ERICK MACHADO RODRIGUES

FELIPE BATISTA DOS PASSOS MELO

GABRIEL ALBERTO DUARTE DE ANDRADE

Brasília 2026

ANÁLISE ARQUITETURAL DO DESASTRE DE CHERNOBYL E PROPOSTA DE SOLUÇÃO MODERNA PARA 2026

Trabalho acadêmico apresentado à Universidade Católica de Brasília, no curso de engenharia de software, como requisito avaliativo da disciplina de Arquitetura de Software / Arquitetura de Software.

Professor(a): Jefferson salomão

RESUMO

O presente trabalho analisa o desastre de Chernobyl, ocorrido em 26 de abril de 1986, sob a perspectiva da Arquitetura de Software e da Engenharia de Requisitos. A pesquisa parte do entendimento de que, em sistemas críticos, falhas humanas e operacionais não podem ser dissociadas das decisões arquiteturais, da ausência de mecanismos de monitoramento, da baixa resiliência, da insuficiência de barreiras de segurança e da priorização inadequada de requisitos. Com base em literatura técnica e fontes institucionais, descreve-se o contexto histórico do acidente, seus impactos humanos, sociais, econômicos e ambientais, bem como as limitações de projeto e de operação do reator RBMK envolvido. Em seguida, são identificados requisitos funcionais e não funcionais ausentes ou mal priorizados, como validação automática de condições inseguras, rastreabilidade, tolerância a falhas, observabilidade e resposta automatizada a eventos críticos. Como contribuição principal, propõe-se uma solução arquitetural moderna com visão para 2026, adotando arquitetura híbrida composta por camadas N-tier, microsserviços e comunicação orientada a eventos. Também são indicados padrões de projeto capazes de reduzir acoplamento, ampliar confiabilidade e melhorar resposta a incidentes, como Observer, Strategy, Factory e Circuit Breaker. Por fim, apresenta-se uma simulação em Python, com dados mockados, destinada a demonstrar como um sistema reestruturado poderia monitorar sensores, validar limites operacionais, emitir alertas e interromper procedimentos inseguros. Conclui-se que a Arquitetura de Software, quando tratada de forma estratégica em sistemas críticos, tem papel central na prevenção de acidentes de grande escala.

Palavras-chave: Chernobyl; Arquitetura de Software; Sistemas críticos; Engenharia de Requisitos; Resiliência.

SUMÁRIO

1 INTRODUÇÃO **5**
2 EXPLICAÇÃO DO PROBLEMA (DESASTRE) **5**
3 ANÁLISE DO PROBLEMA SOB A PERSPECTIVA DA ARQUITETURA DE SOFTWARE **7**
4 ANÁLISE ESSENCIAL DO SISTEMA **8**
5 ANÁLISE DE REQUISITOS **12**
5.1 Requisitos faltantes 12
5.2 Requisitos mal priorizados 13
6 PROPOSTA DE SOLUÇÃO ARQUITETURAL (VISÃO 2026) **14**
6.1 Estilo arquitetural adotado 14
6.2 Estrutura proposta 14
7 DESIGN PATTERNS APLICADOS **15**
8 MODELAGEM DA SOLUÇÃO **16**
9 IMPLEMENTAÇÃO DA SOLUÇÃO (SIMULAÇÃO EM PYTHON) **17**
10 DISCUSSÃO E CONCLUSÃO **18**
REFERÊNCIAS **18**
APÊNDICE A – ORIENTAÇÕES PARA USO DA SIMULAÇÃO **19**

1 INTRODUÇÃO

O acidente de Chernobyl, ocorrido em 26 de abril de 1986, constitui um dos episódios mais emblemáticos da história dos sistemas críticos. Embora normalmente seja tratado como um desastre nuclear provocado por uma combinação de erro humano, decisões operacionais inadequadas e falhas de projeto do reator, o evento também pode ser reinterpretado como um caso paradigmático de insuficiência arquitetural em um sistema sociotécnico de alta complexidade. Essa leitura é pertinente porque o funcionamento seguro de uma usina nuclear depende não apenas de componentes físicos, mas da integração coerente entre sensores, sistemas de controle, procedimentos, barreiras automáticas, alertas, supervisão humana e resposta emergencial.

A proposta deste trabalho consiste em analisar Chernobyl sob a ótica da Arquitetura de Software e da Engenharia de Requisitos, identificando limitações estruturais que contribuíram para a escalada do acidente e propondo uma solução arquitetural moderna, compatível com práticas e tecnologias disponíveis em 2026. O enfoque está alinhado ao enunciado da atividade, que solicita a análise crítica de um desastre histórico, a revisão de requisitos, a modelagem do domínio, a proposição de uma arquitetura contemporânea e a implementação parcial de uma simulação.

Além de cumprir o objetivo acadêmico, a discussão é relevante porque evidencia que, em sistemas críticos, escolhas arquiteturais não são meramente técnicas: elas influenciam diretamente a capacidade de prevenção, contenção e recuperação diante de eventos extremos. Dessa forma, o caso de Chernobyl fornece base consistente para discutir acoplamento excessivo, baixa resiliência, ausência de observabilidade, falhas de segurança e deficiências de governança operacional.

2 EXPLICAÇÃO DO PROBLEMA (DESASTRE)

Na madrugada de 26 de abril de 1986, durante um teste em baixa potência no reator 4 da Usina Nuclear de Chernobyl, houve perda de controle do sistema, seguida por explosão e incêndio que destruíram o edifício do reator e liberaram grande quantidade de material radioativo na atmosfera. A literatura técnica consolidada aponta

que o acidente ocorreu em um contexto de teste inadequadamente conduzido, com medidas de segurança ignoradas, além de limitações importantes no projeto do reator RBMK e em seus mecanismos de proteção (IAEA, 2026; NRC, 2026).

Do ponto de vista humano e social, o acidente provocou mortes imediatas entre trabalhadores e equipes de emergência, lesões por radiação aguda, evacuação em massa e deslocamento prolongado de populações inteiras. Segundo o UNSCEAR, cerca de 115 mil pessoas foram evacuadas da área próxima ao reator em 1986 e, posteriormente, outras 220 mil foram reassentadas em Belarus, Federação Russa e Ucrânia. O mesmo órgão destaca ainda o forte impacto social, psicológico e econômico do desastre sobre as regiões afetadas (UNSCEAR, 2026).

Na perspectiva de saúde pública, a Organização Mundial da Saúde ressalta que as consequências do acidente exigiram revisão extensa da literatura científica e programas específicos de assistência, com destaque para os estudos sobre câncer de tireoide em indivíduos expostos ainda na infância ou adolescência (WHO, 2006). Assim, ainda que Chernobyl tenha sido um desastre materialmente nuclear, seu estudo exige abordagem sociotécnica abrangente, envolvendo projeto, operação, monitoramento, tomada de decisão, comunicação de risco e resposta emergencial.

Tabela 1 – Síntese do desastre de Chernobyl

Aspecto	**Síntese**
Data	26 de abril de 1986.
Local	Reator 4 da Usina Nuclear de Chernobyl, então URSS (atual Ucrânia).
Situação geradora	Teste em baixa potência com perda de controle operacional.
Consequência imediata	Explosão, incêndio e liberação maciça de material radioativo.
Impactos ampliados	Mortes, evacuações, contaminação ambiental, danos econômicos e efeitos psicossociais duradouros.

Fonte: elaborado com base em IAEA (2026), UNSCEAR (2026) e NRC (2026).

3 ANÁLISE DO PROBLEMA SOB A PERSPECTIVA DA ARQUITETURA DE SOFTWARE

A interpretação arquitetural de Chernobyl parte do entendimento de que sistemas críticos devem ser projetados para operar com segurança mesmo quando há erro humano, ambiguidade operacional ou condições degradadas. Em outras palavras, uma arquitetura robusta precisa restringir ações inseguras, oferecer observabilidade ampla, degradar de forma controlada e acionar barreiras automáticas antes que a falha se torne catastrófica. Sob esse prisma, o desastre revela forte assimetria entre a criticidade do domínio e a capacidade do sistema de conter comportamentos perigosos.

O relatório INSAG-7, publicado pela IAEA, é especialmente relevante por reconhecer que a explicação inicial centrada apenas em violações operacionais era insuficiente. A revisão posterior passou a enfatizar também deficiências do projeto do reator e do sistema de controle e proteção, indicando que o acidente não pode ser atribuído exclusivamente ao comportamento da equipe de operação (IAEA, 1992). Essa constatação é fundamental para o debate arquitetural, pois demonstra que a estrutura do sistema não oferecia proteção adequada contra decisões indevidas em contexto crítico.

Do ponto de vista da Arquitetura de Software, podem ser observadas pelo menos cinco classes de falhas. A primeira é o acoplamento excessivo entre operação humana, estado físico do reator e mecanismos de proteção, sem isolamento suficiente entre subsistemas críticos. A segunda é a baixa resiliência: o sistema não foi capaz de retornar rapidamente a um estado seguro nem de degradar de modo previsível. A terceira é a ausência de observabilidade adequada, o que limita a compreensão situacional dos operadores. A quarta é a insuficiência de tolerância a falhas e bloqueios preventivos. A quinta é a baixa capacidade de monitoramento inteligente e de resposta orientada a eventos.

Em termos contemporâneos, trata-se de um sistema com arquitetura pouco defensiva. Em um domínio no qual a segurança deveria prevalecer sobre metas operacionais locais, seria esperado que o software e os mecanismos de automação barrassem testes em configurações perigosas, mantivessem trilhas de auditoria

robustas, emitissem alertas inequívocos e reduzissem a dependência de julgamento humano sob pressão.

Tabela 2 – Falhas arquiteturais observáveis no caso Chernobyl

Falha	Descrição	Impacto no desastre
Acoplamento excessivo	Dependência elevada entre ações humanas, condições do reator e resposta dos mecanismos de proteção.	Ampliação do risco sistêmico e dificuldade de isolamento da falha.
Baixa resiliência	Inexistência de recuperação rápida para estado seguro em condição anormal.	Escalada acelerada do evento crítico.
Observabilidade limitada	Visão insuficiente do estado real do sistema em tempo de operação.	Tomada de decisão com informação incompleta.
Ausência de bloqueios automáticos fortes	Regras de segurança não impediram adequadamente a continuidade do teste.	Permissão de ações incompatíveis com operação segura.
Monitoramento pouco reativo	Detecção e tratamento de anomalias sem arquitetura orientada a eventos críticos.	Resposta tardia diante da degradação operacional.

Fonte: elaborado pelos autores com base em IAEA (1992; 2026).

4 ANÁLISE ESSENCIAL DO SISTEMA

A análise essencial busca abstrair o sistema do ponto de vista do domínio, independentemente da tecnologia empregada. Nesse nível, o problema central consiste em garantir operação segura do reator por meio da coleta contínua de sinais, interpretação de limites operacionais, bloqueio de procedimentos inseguros, comunicação de eventos críticos e coordenação de resposta emergencial.

Sob essa ótica, as principais entidades do domínio podem ser descritas como: Reator, Sensor, Leitura de Sensor, Operador, Supervisor, Teste de Segurança, Alerta, Incidente, Protocolo de Emergência e Log de Auditoria. Os relacionamentos entre essas entidades sustentam os diagramas exigidos no trabalho: DER, Caso de Uso, Diagrama de Classes e Diagrama de Atividades. A seguir, apresentam-se os diagramas elaborados com base nessa estrutura conceitual, mantendo coerência com o domínio analisado.

Os casos de uso essenciais incluem monitorar parâmetros do reator, validar condições de teste, autorizar procedimento, bloquear ação insegura, emitir alerta crítico, registrar incidente e acionar protocolo de emergência. Em um diagrama de atividades, o fluxo central começa pela solicitação do teste, prossegue com a leitura dos sensores e validação das condições, e se divide entre continuidade controlada da operação ou interrupção imediata com resposta emergencial quando há violação de limite seguro.

Tabela 3 – Entidades essenciais para modelagem UML

Entidade	**Papel no domínio**
Reator	Elemento físico monitorado e protegido pelo sistema.
Sensor	Origem dos dados operacionais, como temperatura, pressão e potência.
LeituraSensor	Registro temporal dos valores capturados.
Operador	Agente humano que inicia ou acompanha procedimentos.
Supervisor	Responsável por autorizações e acompanhamento de criticidade.
Alerta	Objeto de notificação quando limites são excedidos ou risco é detectado.
Incidente	Evento anômalo registrado para auditoria e resposta.
ProtocoloEmergencia	Conjunto de ações disparadas automaticamente ou por validação humana.

Fonte: elaborado pelos autores.

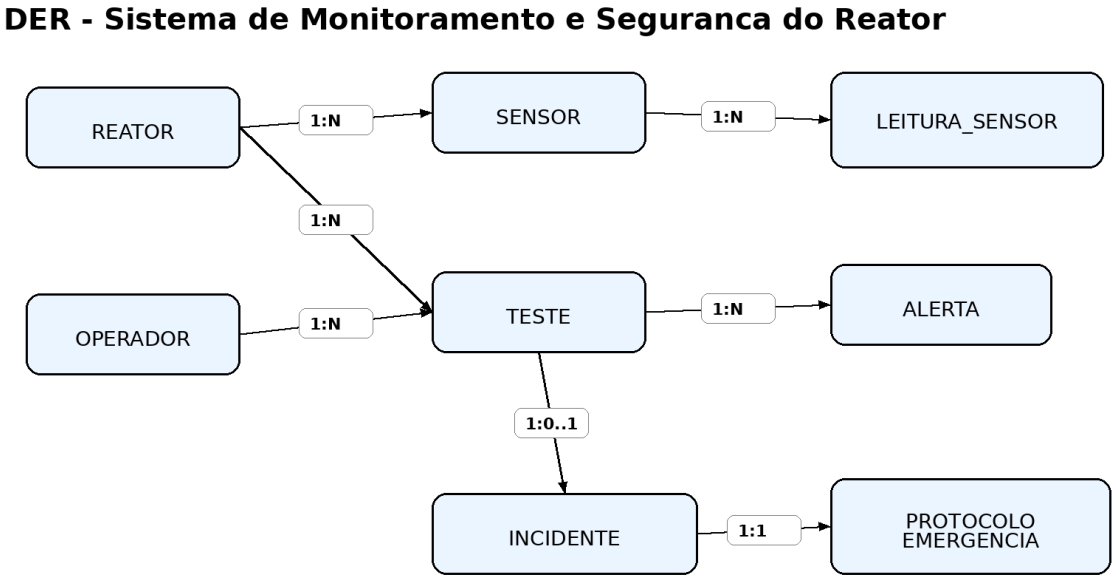

Figura 1 - DER do sistema de monitoramento e segurança do reator

Fonte: elaborado pelos autores.

Diagrama de Casos de Uso - Sistema de Seguranca do Reator

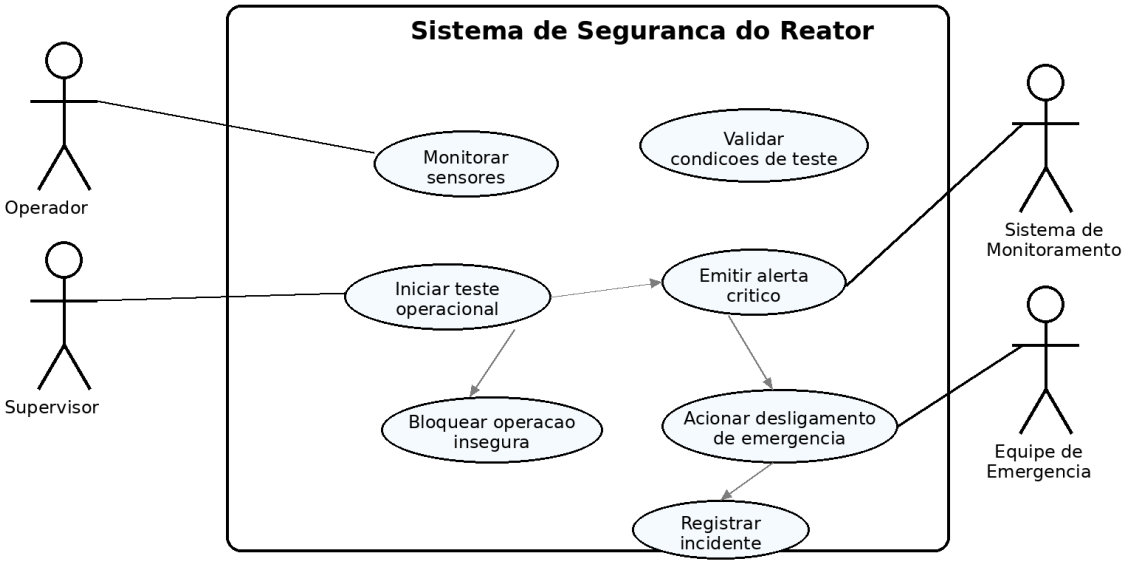

Figura 2 - Diagrama de casos de uso do sistema proposto

Fonte: elaborado pelos autores.

Diagrama de Classes - Dominio do Sistema

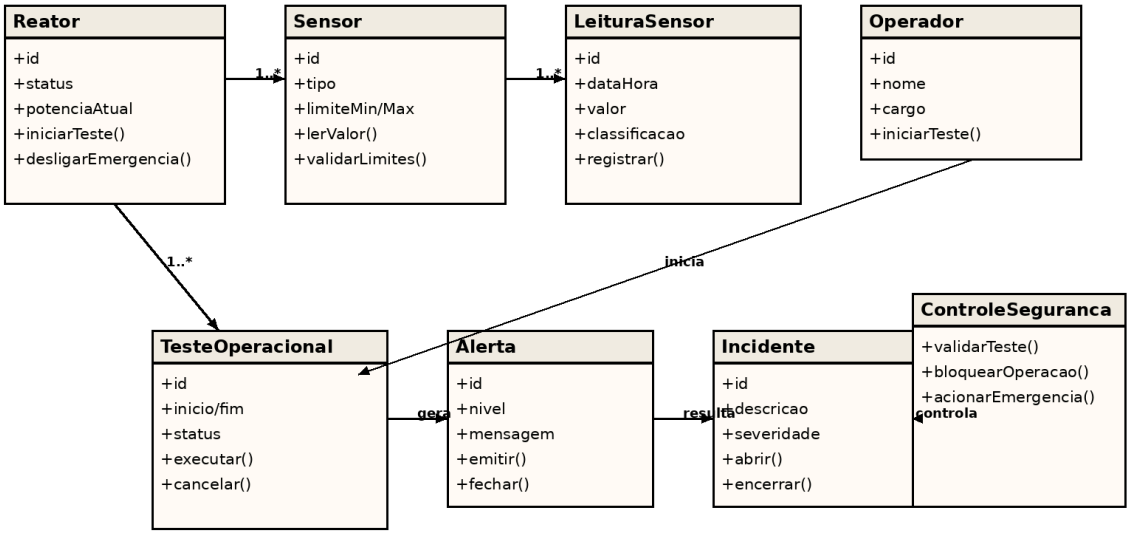

Figura 3 - Diagrama de classes do dominio

Fonte: elaborado pelos autores.

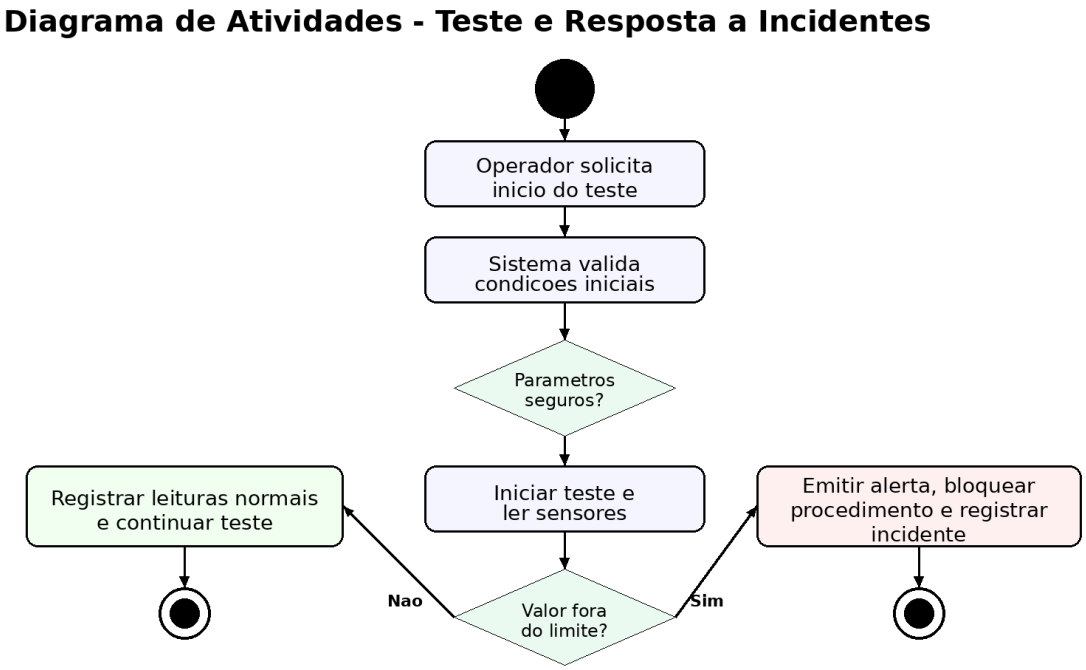

Figura 4 - Diagrama de atividades do processo de teste e resposta a incidentes

Fonte: elaborado pelos autores.

5 ANÁLISE DE REQUISITOS

5.1 Requisitos faltantes

A revisão do caso indica a ausência ou insuficiência de requisitos que hoje seriam considerados indispensáveis em sistemas críticos. Entre os requisitos funcionais, destacam-se: validação automática das condições mínimas de segurança antes da execução de qualquer teste; bloqueio compulsório de procedimentos incompatíveis com o estado do reator; emissão multicanal de alertas; registro centralizado de eventos; e acionamento automático de protocolos de emergência.

Quanto aos requisitos não funcionais, sobressaem disponibilidade, confiabilidade, rastreabilidade, auditabilidade, segurança operacional, observabilidade, baixa latência para reação a eventos críticos e tolerância a falhas. A ausência desses requisitos reduz a capacidade do sistema de prevenir acidentes, de explicá-los durante sua ocorrência e de reconstruir os fatos posteriormente para aprendizagem organizacional.

5.2 Requisitos mal priorizados

O caso também sugere conflitos de priorização entre objetivos operacionais e metas de segurança. Em contextos dessa natureza, qualquer prioridade dada à continuidade do teste, ao cumprimento do cronograma ou à execução local da tarefa em detrimento de barreiras de segurança representa erro de engenharia. A análise contemporânea aponta que, em sistemas críticos, segurança, confiabilidade e explicabilidade devem prevalecer sobre metas operacionais secundárias. Isso significa que o sistema precisa ser construído para dizer não, inclusive ao próprio operador, sempre que os parâmetros seguros não forem satisfeitos.

Tabela 4 – Requisitos ausentes ou mal priorizados

Tipo	Requisito	Consequência da ausência ou baixa prioridade
Funcional	Validação automática pré-teste	Permite iniciar procedimento em contexto inseguro.
Funcional	Bloqueio de continuidade	Não impede escalada após detecção de anomalia.
Não funcional	Observabilidade	Reduz clareza situacional e piora decisões sob pressão.
Não funcional	Auditabilidade	Dificulta rastrear causas, responsabilidades e lições.
Não funcional	Tolerância a falhas	Aumenta chance de propagação de estados perigosos.

Tipo	Requisito	Consequência da ausência ou baixa prioridade
Priorização	Segurança acima da operação	Sua inversão amplia a vulnerabilidade do sistema.

Fonte: elaborado pelos autores.

6 PROPOSTA DE SOLUÇÃO ARQUITETURAL (VISÃO 2026)

6.1 Estilo arquitetural adotado

Para reimaginar a solução em 2026, propõe-se uma arquitetura híbrida composta por organização em camadas N-tier, microsserviços para responsabilidades críticas e comunicação orientada a eventos. A justificativa para essa escolha reside no fato de que sistemas de alta criticidade precisam combinar separação de responsabilidades, escalabilidade, auditabilidade e reação imediata a eventos anômalos. Uma única arquitetura monolítica, embora possa simplificar implantação em alguns cenários, tenderia a dificultar isolamento de responsabilidades e evolução independente de módulos sensíveis.

6.2 Estrutura proposta

Na camada de apresentação, dashboards distintos atenderiam operadores, supervisores e equipes de resposta. Na camada de aplicação, seriam implementados orquestradores de teste, serviços de validação e controladores de política operacional. Na camada de domínio, ficariam as regras de segurança, os limites operacionais, os critérios de bloqueio e os protocolos de resposta. Na camada de infraestrutura, estariam os mecanismos de persistência, mensageria, logging, integração com sensores e serviços externos de notificação.

Em paralelo, alguns serviços seriam explicitamente desacoplados como microsserviços: Serviço de Monitoramento, Serviço de Validação de Segurança, Serviço de Alertas, Serviço de Auditoria e Serviço de Resposta Emergencial. A comunicação entre eles ocorreria por meio de eventos, de modo que leituras críticas ou estados

inválidos gerassem mensagens imediatamente processadas por consumidores especializados. Isso favorece reação rápida, rastreabilidade e desacoplamento temporal entre componentes.

Tabela 5 – Proposta arquitetural para 2026

Camada / Serviço	Responsabilidade principal
Apresentação	Exibir dashboards, alertas, autorização e histórico para perfis diferentes.
Aplicação	Orquestrar testes, iniciar fluxos e coordenar serviços.
Domínio	Concentrar regras de segurança, limites e políticas de bloqueio.
Infraestrutura	Persistir logs, publicar eventos, integrar sensores e notificações.
Serviço de Monitoramento	Receber e consolidar leituras em tempo real.
Serviço de Validação	Comparar leituras com limites operacionais e estados válidos.
Serviço de Alertas	Emitir notificações críticas para operadores e supervisores.
Serviço de Auditoria	Registrar trilhas de decisão e incidentes para posterior análise.

Fonte: elaborado pelos autores.

7 DESIGN PATTERNS APLICADOS

Alguns padrões de projeto se mostram particularmente adequados à solução proposta. O padrão Observer permite que sensores e módulos de monitoramento notifiquem automaticamente componentes interessados quando ocorre alteração crítica em determinada variável. O padrão Strategy pode ser utilizado para definir diferentes respostas conforme o tipo de incidente: sobretemperatura, perda de resfriamento,

instabilidade de potência ou falha de comunicação. O padrão Factory facilita a criação padronizada de alertas, mensagens de evento e objetos de protocolo emergencial. Já o Circuit Breaker contribui para impedir que falhas em serviços periféricos comprometam o núcleo das decisões de segurança. Em contexto de configuração crítica, um Singleton controlado também pode ser empregado para manter parâmetros versionados e centralizados, desde que combinado com mecanismos adequados de auditoria.

Mais importante do que citar padrões isoladamente é demonstrar sua contribuição prática. No cenário em análise, tais padrões reduzem acoplamento, favorecem evolução independente dos componentes, ampliam previsibilidade e fortalecem o princípio de fail-safe, segundo o qual o sistema deve tender ao estado seguro quando perde capacidade de operar normalmente.

8 MODELAGEM DA SOLUÇÃO

Para a modelagem da solução, apresenta-se na Figura 5 um diagrama N-tier representando as camadas de apresentação, aplicação, domínio e infraestrutura. Em complemento, a análise essencial do sistema contempla os diagramas UML obrigatórios definidos no enunciado da atividade, agora incorporados ao documento. A coerência entre essas representações é preservada por meio de nomenclatura consistente das entidades do domínio, eventos, serviços e fluxos de decisão.

Um fluxo de sequência típico seria o seguinte: o operador solicita início do teste; o orquestrador consulta o serviço de monitoramento; o serviço de validação compara leituras com limites; se houver anomalia, um evento crítico é publicado; o serviço de alertas notifica os responsáveis; o serviço de resposta emergencial bloqueia o procedimento; e o serviço de auditoria grava o incidente com marca temporal, causa presumida e ações executadas. Essa estrutura permite traduzir o raciocínio arquitetural em diagramas e, posteriormente, em implementação simulada.

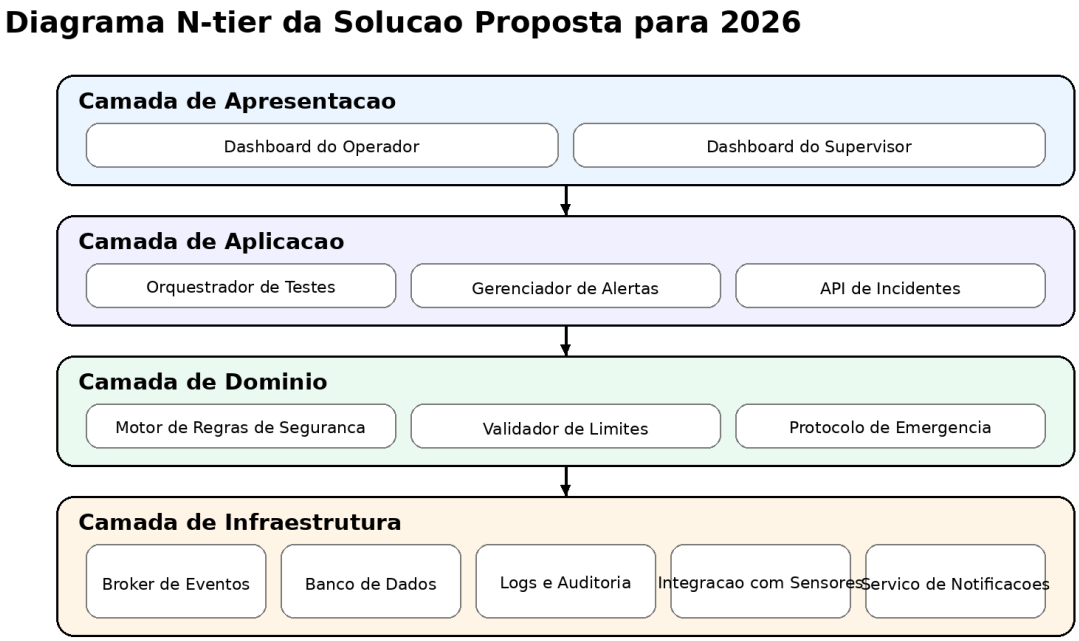

Figura 5 - Diagrama N-tier da arquitetura proposta para 2026

Fonte: elaborado pelos autores.

9 IMPLEMENTAÇÃO DA SOLUÇÃO (SIMULAÇÃO EM PYTHON)

Conforme solicitado na atividade, a implementação pode ser parcial e simulada, com dados mockados e foco na estrutura arquitetural. Para atender a esse requisito, foi preparado um script em Python separado deste documento, denominado “chernobyl_simulacao.py”. O programa representa de forma simplificada a arquitetura proposta: sensores mockados geram leituras de temperatura, pressão e potência; um validador compara os valores com limites seguros; um gerenciador de alertas emite mensagens; e um registrador de incidentes persiste o histórico em arquivo JSON.

A lógica central da simulação é demonstrar que, em um sistema arquitetado de forma mais robusta, a operação insegura não prosseguiria sem contestação. Ao detectar valor fora do limite, o script interrompe o teste, registra o incidente, aciona um protocolo de emergência simulado e informa claramente o motivo do bloqueio. Embora simples, essa implementação materializa a proposta de arquitetura defendida neste

trabalho e pode ser expandida para interfaces gráficas, banco de dados e comunicação assíncrona em versões futuras.

10 DISCUSSÃO E CONCLUSÃO

A releitura de Chernobyl sob a ótica da Arquitetura de Software permite compreender que acidentes de grande escala não decorrem apenas de atos isolados, mas da interação entre projeto, operação, informação disponível, prioridades institucionais e mecanismos de contenção. Um sistema crítico adequadamente arquitetado precisa presumir a possibilidade de erro humano e, por isso, deve incorporar redundância, observabilidade, fail-safe, validação forte e capacidade de resposta automatizada.

A solução proposta para 2026 não pretende sugerir que software, por si só, eliminaria todo risco nuclear. O argumento central é outro: decisões arquiteturais mais maduras poderiam reduzir significativamente a probabilidade de escalada catastrófica, melhorar a detecção precoce de estados inseguros, tornar a operação mais transparente e fortalecer a responsabilização organizacional. Nesse sentido, a contribuição deste trabalho está em demonstrar como princípios contemporâneos de arquitetura e requisitos podem ser aplicados a um desastre histórico, produzindo aprendizado técnico relevante para sistemas críticos atuais.

Conclui-se, portanto, que a Arquitetura de Software deve ser tratada como dimensão estratégica de segurança. Em ambientes de alta criticidade, ela não se limita à organização de código ou infraestrutura: constitui parte essencial da defesa em profundidade do sistema. O caso Chernobyl segue sendo, décadas depois, um exemplo contundente de que falhas arquiteturais e de requisitos podem amplificar dramaticamente as consequências de uma operação malsucedida.

REFERÊNCIAS

INTERNATIONAL ATOMIC ENERGY AGENCY. *The 1986 Chornobyl nuclear power plant accident*. Vienna: IAEA, 2026. Disponível em: <https://www.iaea.org/topics/chornobyl>. Acesso em: 6 abr. 2026.

INTERNATIONAL ATOMIC ENERGY AGENCY. *The Chernobyl Accident: Updating of INSAG-1*. Vienna: IAEA, 1992. Disponível em: <https://www-pub.iaea.org/MTCD/Publications/PDF/Pub913e_web.pdf>. Acesso em: 6 abr. 2026.

NUCLEAR REGULATORY COMMISSION. *Backgrounder on Chernobyl Nuclear Power Plant Accident*. Washington: NRC, 2026. Disponível em: <https://www.nrc.gov/reading-rm/doc-collections/fact-sheets/chernobyl-bg>. Acesso em: 6 abr. 2026.

UNIVERSIDADE CATÓLICA DE BRASÍLIA. *Manual para apresentação de monografias, dissertações e teses da Universidade Católica de Brasília*. 21. ed. Brasília: UCB, 2025. Disponível em: <https://ucb2.catolica.edu.br/portal/wp-content/uploads/2025/09/Manual_21ed_.pdf>. Acesso em: 6 abr. 2026.

UNITED NATIONS SCIENTIFIC COMMITTEE ON THE EFFECTS OF ATOMIC RADIATION. *The Chornobyl Accident*. New York: UNSCEAR, 2026. Disponível em: <https://www.unscear.org/unscear/en/areas-of-work/chernobyl.html>. Acesso em: 6 abr. 2026.

WORLD HEALTH ORGANIZATION. *Health effects of the Chernobyl accident: an overview*. Geneva: WHO, 2006. Disponível em: <https://www.who.int/docs/default-source/documents/publications/health-effects-of-the-chernobyl-accident.pdf>. Acesso em: 6 abr. 2026.

APÊNDICE A – ORIENTAÇÕES PARA USO DA SIMULAÇÃO

https://github.com/Gabrielalbertodu/projeto-arquitetura-de-software-Chernobyl

1. Instale Python 3.10 ou superior. 2. Salve o arquivo “chernobyl_simulacao.py” na mesma pasta em que deseja gerar o log do incidente. 3. Execute o comando “chernobyl_simulacao.py”. 4. Observe, no terminal, a sequência de leituras, a validação dos limites e a interrupção do teste quando a condição insegura for detectada. 5. Após a execução, verifique o arquivo “incident_log.json”, criado automaticamente pelo script.

Observação: por se tratar de simulação didática, os dados são mockados e as leituras foram escolhidas para demonstrar um cenário de degradação operacional com posterior acionamento do protocolo de emergência.